

PROGRAMMING FUNDAMENTALS PROJECT

Report

Submitted To: Sir Adeel Khalid

Submitted By

Group Members

Saniya Rizwan: 2023-BCS-079

Kanwal Noor ul Ain: 2023-BCS-049

Hajra Iftikhar: 2023-BCS-035

Memona Bibi: 2023-BCS-055

Natasha Maryam: 2023-BCS-068

Hifsa Kanwal: 2023-BCS-039

Table of Contents

Mathematical Problem-Solving Game.....	3
Background	3
Problem Statement.....	3
Introduction.....	3
Methodology	4
Conclusion	5
References.....	6
Appendices.....	6

Mathematical Problem-Solving Game

Background

The background of the program lies in the intersection of education, gaming, and programming. The Mathematical Problem-Solving Game is a simple console-based program that is designed to comprehensively test and improve users' arithmetic skills. The program is developed in C which successfully incorporates random number generation, basic arithmetic operations, and user input to create an interactive and user-friendly learning experience with the help of a mathematical game. By presenting a series of random arithmetic problems, users are encouraged to apply their knowledge of addition, subtraction, multiplication, and division in a dynamic manner. The emergence of digital tools has somehow led to a diminishing emphasis on foundational arithmetic skills and this game seeks to bridge the gap by providing a platform that encourages the users to proactively participate in solving mathematical problems. The main purpose of the game is to provide users with a challenging yet enjoyable way to enhance and improve their mathematical abilities.

Problem Statement

The declining rate of mathematical proficiency among individuals, particularly in basic arithmetic, has become a growing concern. As the technology sector is growing rapidly the dependence on digital calculators and digital tools has increased which potentially leads to a deterioration of fundamental math skills. Also, traditional methods of teaching mathematics often struggle to maintain student engagement. Digital tools and calculators are valuable in certain contexts, and they may contribute to a decline in individuals' ability to perform mental arithmetic and comprehend fundamental numerical concepts. Moreover, while educational technology has made significant progress there is often a gap in the development of engaging tools for reinforcing basic skills. The mathematical Problem-Solving Game addresses this issue by offering a platform that encourages users to practice and strengthen their arithmetic abilities interactively.

Introduction

The Mathematical Problem Solving Game is designed to be a simple yet effective tool for users of all ages to enhance their arithmetic skills. The program developed in the C programming language, redefines the traditional approach to learning mathematics by transforming it into a captivating and interactive experience. The program aims to provide an educational and entertaining experience by combining elements of randomness, arithmetic challenges, and a

scoring system to keep users engaged in solving mathematical problems (Seidametova, 2020). The purpose of the game is to engage the users in a series of randomly generated mathematical questions, challenging them to solve equations involving addition, subtraction, multiplication, and division. Furthermore, the difficulty level increases as the user progresses which provides a continuous and adaptive learning experience. This report comprehensively covers information on the development of a mathematical problem-solving game in a console-based application in C language.

Methodology

The game employs a simple and easier console-based interface using C programming language. It incorporates random number generation for problem creation, basic arithmetic operations (addition, subtraction, multiplication, and division), and different levels of difficulty. However, the difficulty level increases as the player successfully answers a set number of questions (Martins, et al., 2018). The game continues as soon as the user provides correct and the program exists whenever the incorrect answer is entered or user decides to exit. The program involves the usage of the following functions:

- **Random Number Generation:**

The program utilizes the 'rand()' function along with a seed based on the current time to generate random numbers. This ensures a varied set of questions with each playthrough. (as shown in Appendix A)

```
srand(time(NULL)); // Seed for random number generation
```

- **Question Generation:**

Each question involves two random numbers and an operator (+,-,*,or /) and the difficulty level increases every five correct answers, thus, scaling the range of random numbers. (as shown in the Appendix B)

```
int level = score / 5 + 1; // Increase level every 5 correct answers
```

```
int num1 = generateRandomNumber(level * 10);
```

```
int num2 = generateRandomNumber(level * 10);
```

```
int operator = generateRandomOperator();
```

- **User Input and Validation:**

The program prompts the user to solve the generated mathematical question and it accepts the user input, validates it, and compares it with the correct answer. The game

continues until the user decides to exit or provides an incorrect answer. (as shown in Appendix C)

```
scanf("%s", input);
```

```
userAnswer = atoi(input);
```

- **Scoring and Feedback:**

The user's score is incremented for each correct answer, and feedback is provided to encourage participation. Incorrect answers prompt the display of the correct solution along with the final score. (as shown in the Appendix D)

```
if (userAnswer == correct) {
```

```
    printf("Correct! Your current score: %d\n\n", ++score);
```

```
} else {
```

```
    printf("Incorrect. The correct answer is %d. Your final score: %d\n", correct, score);
```

```
    break;
```

```
}
```

Conclusion

The mathematical problem-solving game presents a creative solution to address the decreasing mathematical proficiency by providing a platform that combines education with entertainment. Through leveraging the principles of gamification, the program transforms learning into an interactive experience, making it more likely for users to invest time and effort. The program's simplicity allows users of all ages to participate, making it an inclusive tool for educational purposes. The adaptive difficulty level ensures that users are consistently challenged, preventing boredom and maintaining interest.

References

Martins, V., de Almeida Souza Concilio, I. & de Paiva Guimarães, M., 2018. Problem based learning associated to the development of games for programming teaching. *Computer Applications in Engineering Education*, 26(5), pp. 1577-1589.

Seidametova, Z., 2020. Combining Programming and Mathematics through Computer Simulation Problems. *ICTERI Workshops*, Volume 2732, pp. 869-880.

Appendices

Appendix A

```
int main() {  
    srand(time(NULL)); // Seed for random number generation
```

Appendix B

```
while (1) {  
    int level = score / 5 + 1; // Increase level every 5 correct answers  
    int num1 = generateRandomNumber(level * 10);  
    int num2 = generateRandomNumber(level * 10);  
    int operator = generateRandomOperator();
```

Appendix C

```
scanf("%s", input);  
  
// Check if the user wants to exit  
if (strcmp(input, "exit") == 0) {  
    printf("Exiting the game. Your final score: %d\n", score);  
    break;  
}  
  
// Convert user input to integer  
userAnswer = atoi(input);
```

Appendix D

```
// Check the answer  
if (userAnswer == correct) {  
    printf("Correct! Your current score: %d\n\n", ++score);  
}  
else {  
    printf("Incorrect. The correct answer is %d. Your final score: %d\n", correct, score);  
    break;  
}
```